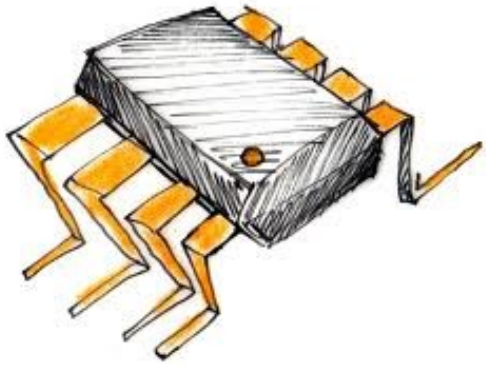
MIKROPROCESORSKI UPRAVLJAČKI UREĐAJI



Praktikum za laboratorijske vežbe

Vežba 1

Profesor:
dr Vojin Ilić

Asistenti:
Bojan Jorgovanović
Ksenija Baraković
Anja Vranješević

Vežba 1

Cilj vežbe:

- Podsećanje na programski jezik C.
- Upoznavanje sa okruženjem Keil µVision. Programe testirati simulatorom za 8051.

Zadatak 1:

Napisati C program koja u niz dužine n smešta prvih n parnih brojeva.

Statička i dinamička alokacija memorije

Kod **statičke alokacije** veličina memorije je poznata već u trenutku prevođenja programa. Prevodilac unapred rezerviše prostor za promenljivu ili niz, a taj prostor se ne može menjati tokom izvršavanja programa.

Kod **dinamičke alokacije** memorija se zauzima tokom izvršavanja programa, pomoću funkcija iz biblioteke `<stdlib.h>` (`malloc`, `calloc`, `realloc`). Veličina može zavisiti od podataka poznatih tek u toku rada (npr. od unosa korisnika), ali programer mora sam da oslobodi zauzetu memoriju funkcijom `free`.

```
#include <stdlib.h>

int niz_s[10];                    // statička alokacija: uvek 10 elemenata

void primer(int n)
{
    int *niz_d;
    niz_d = malloc(n * sizeof(int));  // dinamička alokacija: n elemenata
    if (niz_d != NULL)                // provera da li je alokacija uspela
    {
        niz_d[0] = 2;
        // ...
        free(niz_d);                  // oslobađanje memorije
    }
}
```

Osnovne razlike:

- **Trenutak zauzimanja:** statička – pri prevođenju; dinamička – tokom izvršavanja.
- **Veličina:** statička je fiksna; dinamička može zavisiti od podataka u toku rada.
- **Oslobađanje:** statičku memoriju oslobađa sam sistem; dinamičku mora osloboditi programer (inače dolazi do curenja memorije).
- **Pouzdanost:** statička alokacija uvek uspeva ako program može da se prevede; dinamička može da ne uspe (`malloc` vraća `NULL`).

Mikrokontroleri kao što je AT89C51RC2 imaju relativno malo RAM memorije, pa se kod njih dinamička alokacija uglavnom izbegava. Zbog toga se u ovim vežbama koriste statički nizovi čija je maksimalna dužina unapred određena.

Biblioteka stdint.h

Veličina osnovnih tipova u jeziku C (`int`, `long`, ...) nije ista na svim platformama. Na primer, `int` na arhitekturi 8051 zauzima 16 bita, dok na personalnom računaru obično zauzima 32 bita. Kod mikrokontrolera je često bitno tačno znati koliko bita ima promenljiva, jer ona odgovara veličini registra ili porta.

Biblioteka `<stdint.h>` definiše celobrojne tipove fiksne veličine:

- `int8_t`, `int16_t`, `int32_t` – označeni brojevi od 8, 16 i 32 bita,
- `uint8_t`, `uint16_t`, `uint32_t` – neoznačeni brojevi od 8, 16 i 32 bita.

```
#include <stdint.h>

uint8_t  a = 255;       // 8 bita, opseg 0 .. 255
int8_t   b = -100;      // 8 bita, opseg -128 .. 127
uint16_t c = 60000;     // 16 bita, opseg 0 .. 65535
int32_t  d = -100000;   // 32 bita
uint8_t  s[11] = "Vezba␣1";  // string kao niz od 8-bitnih znakova
                             // (10 karaktera + '\0' na kraju)

uint8_t setBit(uint8_t x, uint8_t n)  // umesto unsigned char
{
    // ...
}
```

Zadatak 2:

Napisati C program koji računa zbir elemenata iznad sporedne dijagonale kvadratne matrice.

Zadatak 3:

Implementirati sledeće C funkcije:

```
uint8_t set(uint8_t x, uint8_t n);
uint8_t reset(uint8_t x, uint8_t n);
uint8_t setN(uint8_t x, uint8_t n);
uint8_t resetN(uint8_t x, uint8_t n);
```

Funkcija `set` postavlja n-ti bit u promenljivoj x na 1.
Funkcija `reset` resetuje n-ti bit u promenljivoj x na 0.
Funkcija `setN` postavlja prvih n bita u promenljivoj x na 1.
Funkcija `resetN` resetuje prvih n bita u promenljivoj x na 0.

Zadatak 4:

Napisati C program koji radi konkatenaciju stringova različitih dužina. Maksimalna dužina stringa je 10 karaktera.

Rad sa stringovima

U programskom jeziku C ne postoji poseban tip za string. String je niz karaktera koji se završava karakterom `'\0'` (bajt vrednosti 0), po kome procesor prepozna gde je kraj stringa. String se može definisati na tri načina:

```
uint8_t string1[10] = {'I', 'M', 'E', '\0'};   // niz karaktera
uint8_t string2[8] = "Prezime";                // niz inicijalizovan literalom
uint8_t* string3 = "Broj indeksa";             // pokazivač na literal
```

- **Niz karaktera** (`string1`) – rezerviše se niz od 10 bajtova u RAM memoriji, a karakteri se navode pojedinačno. Završni karakter `'\0'` se pri ovakvoj definiciji ne dodaje sam, već ga je potrebno navesti ručno, kao poslednji karakter stringa. Na vrednosti preostalih, nekorišćenih bajtova niza ne treba se oslanjati, jer se oni ne postavljaju na 0 na svim uređajima i sa svim prevodiocima. Ako je potrebno da i oni budu 0, treba ih ručno popuniti nulama.
- **Niz inicijalizovan literalom** (`string2`) – literal `"Prezime"` se kopira u niz u RAM memoriji. Kod svakog literala karakter `'\0'` se implicitno nalazi na kraju stringa – prevodilac ga dodaje sam, iako nije napisan. Zato niz mora imati mesta i za njega: `"Prezime"` ima 7 karaktera, a zauzima 8 bajtova.
- **Pokazivač na literal** (`string3`) – ne pravi se niz, već samo pokazivač koji pokazuje na mesto gde se literal `"Broj indeksa"` nalazi u memoriji. I ovde se, kao kod svakog literala, karakter `'\0'` implicitno nalazi na kraju stringa. Prevodilac (npr. Keil za 8051) takve literale smešta u programsku (ROM) memoriju.

Prva dva načina prave **kopiju** stringa u RAM memoriji, pa se njegov sadržaj može menjati tokom izvršavanja programa (npr. `string2[0] = 'p';`). Kod trećeg načina sam string se nalazi u ROM memoriji, u koju program ne može da upisuje, pa **string ne može da se menja** tokom izvršavanja programa – naredba `string3[0] = 'b';` neće promeniti string. Može se promeniti samo pokazivač, tako da pokazuje na neki drugi string. Prednost ovog načina je što string ne zauzima RAM memoriju, koje na mikrokontroleru ima vrlo malo, pa se on koristi za konstantne stringove, npr. fiksne poruke koje uređaj šalje.

Zadatak 5:

Napisati C funkciju `void izracunaj(char* op1, char* op2, char* rez, char operant)` koja u string `rez` upisuje rezultat matematičke operacije (zadate u promenljivoj `operant`) između `op1` i `op2`. Operant može imati vrednosti `'+'`, `'*'`, `'/'`. Operandi mogu biti zadati u binarnoj, decimalnoj ili heksadecimalnoj vrednosti (npr. binarno – `b0101`, decimalno – `153`, heksadecimalno – `HA3`) i predstavljaju 8-bitne promenljive. Napisati program koji testira zadatu funkciju.

NAPOMENA: Zadatak je previše komplikovan da bi se sva logika implementirala u jednoj funkciji. Mnogo je lakše prvo realizovati funkcije `num2str` i `str2num` koje broj pretvaraju u string i obratno.